

République Tunisienne — Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique

Université de Tunis El Manar

Institut Supérieur d'Informatique

Rapport de Mini-Projet

Application de Gestion de Formation

Centre Excellent Training — Green Building

Spring Boot 3.2

Angular 17

MySQL 8

JWT + BCrypt

Stack technique

Java 17 • Spring Security • Hibernate JPA • TypeScript
Angular Material • ng2-charts • Apache POI • reCAPTCHA v2
BCrypt • JWT • JavaMail • Maven • npm

<https://github.com/oussema-abdelmoumen/Gestion-Formation-Excellent-Training>

Réalisé par

Oussema Abdelmoumen & Malek Nasri

Année Universitaire 2025/2026

Contents

Résumé	3
1 Introduction Générale	4
1.1 Contexte et Problématique	4
1.2 Objectifs du Projet	4
1.3 Périmètre fonctionnel	5
2 Architecture et Conception	6
2.1 Architecture Générale	6
2.2 Structure du Backend (Spring Boot)	6
2.3 Structure du Frontend (Angular)	7
3 Modèle de Données	8
3.1 Diagramme Entité-Association	8
3.2 Description des Tables	9
3.3 Script SQL de création	9
4 Sécurité et Authentification	11
4.1 Flux d'Authentification JWT	11
4.2 Implémentation JWT	11
4.3 Configuration Spring Security	12
4.4 Inscription et Vérification Email	13
5 Gestion des Rôles (RBAC)	15
5.1 Les Trois Rôles	15
5.2 Tableau des Permissions Détaillées	15
5.3 Guards Angular	15
6 Implémentation Technique	17
6.1 Entités JPA	17
6.2 Export Excel avec Apache POI	18
6.3 Service Angular avec RxJS	19
7 Contrôles de Saisie et Validation	20
7.1 Validation Backend (Jakarta Bean Validation)	20
7.2 Validation Frontend (Angular)	20
8 Interface Utilisateur	22
8.1 Design System	22
8.2 Pages et Navigation	22
8.3 Fonctionnalités Spéciales	23
8.3.1 Mode Sombre / Mode Clair	23
8.3.2 Tableau de Bord Statistique	23
8.3.3 Panel Formations par Participant	23

9 Tests et Validation Fonctionnelle	24
9.1 Scénarios de Test	24
9.2 Endpoints API testés	24
10 Guide de Déploiement	25
10.1 Prérequis	25
10.2 Configuration	25
10.3 Démarrage	26
11 Captures d’Écran de l’Application	27
11.1 Page d’Accueil (Landing Page)	27
11.2 Pages d’Authentification	27
11.2.1 Page de Connexion	28
11.2.2 Page d’Inscription avec CAPTCHA	28
11.2.3 Email de Vérification	29
11.3 Tableau de Bord (Admin)	30
11.3.1 Vue d’ensemble et Statistiques	30
11.3.2 Barre de Navigation Admin	31
11.4 Dashboard Responsable	32
11.5 Interface Utilisateur Simple	32
12 Conclusion	34
12.1 Bilan du Projet	34
12.2 Perspectives d’Amélioration	34

Résumé

Ce rapport présente la conception et le développement d'une application web complète de gestion de formation réalisée pour le centre **Excellent Training** de la société **Green Building**, dans le cadre du mini-projet de l'ISI Tunis El Manar pour l'année universitaire 2025/2026.

L'application a été développée avec une architecture full-stack moderne :

- **Backend** : Spring Boot 3.2 (Java 17) avec Spring Security, JWT, JPA/Hibernate, MySQL
- **Frontend** : Angular 17 avec TypeScript, Angular Material, ng2-charts
- **Sécurité** : Authentification JWT, BCrypt, reCAPTCHA v2, vérification email

Le projet implémente un système RBAC (Role-Based Access Control) avec trois rôles : Administrateur, Responsable et Utilisateur, chacun ayant des permissions distinctes sur les ressources de l'application.

Chapter 1

Introduction Générale

1.1 Contexte et Problématique

La formation professionnelle continue est un droit fondamental pour tout personnel d'une entreprise ou d'une administration. Elle permet à chaque individu d'acquérir de nouveaux savoirs et savoir-faire, améliorant ainsi ses compétences par des techniques et connaissances actualisées [1].

Le centre de formation **Excellent Training** de la société **Green Building** organisait jusqu'alors ses formations de manière entièrement manuelle :

- Les structures envoyaient leurs besoins en format **papier**
- Les informations étaient saisies dans des **fichiers Excel** créés chaque année
- Absence totale de suivi statistique et d'évaluation automatisée
- Risques élevés d'erreurs humaines et de perte de données

Problèmes identifiés

Perte de temps — Erreurs de saisie — Impossible de faire des statistiques par domaine ou structure — Pas de traçabilité des créateurs de formations — Données éparpillées dans plusieurs fichiers Excel

1.2 Objectifs du Projet

Solution proposée

Développer une application web full-stack permettant de centraliser, sécuriser et automatiser la gestion des formations, avec un contrôle d'accès granulaire par rôles.

- Centraliser toutes les données en base de données MySQL relationnelle
- Implémenter un système RBAC avec 3 niveaux d'accès distincts
- Fournir un tableau de bord statistique interactif
- Assurer la sécurité des données (JWT, BCrypt, CAPTCHA, vérification email)
- Offrir une interface moderne avec dark/light mode et contrôles de saisie rigoureux
- Permettre l'export des données en fichier Excel

1.3 Périmètre fonctionnel

Le projet couvre les cas d'utilisation du cahier des charges [1] :

Table 1.1: Cas d'utilisation principaux

darkblue	Description
Gestion des formations	CRUD complet + organisation (formateur, participants, lieu, date)
Gestion des participants	CRUD avec affiliation structure/profil et vue des formations associées
Gestion des formateurs	CRUD avec distinction interne/externe
Statistiques	Graphiques par domaine, année, budget
Authentification	Login, inscription, vérification email, CAPTCHA
Export Excel	Fichier .xlsx avec mise en forme
Gestion utilisateurs	CRUD comptes (Admin uniquement)

Chapter 2

Architecture et Conception

2.1 Architecture Générale

L'application suit une architecture **Client-Serveur à trois couches** (*3-tier architecture*), communiquant via une **API REST** sécurisée par JWT :

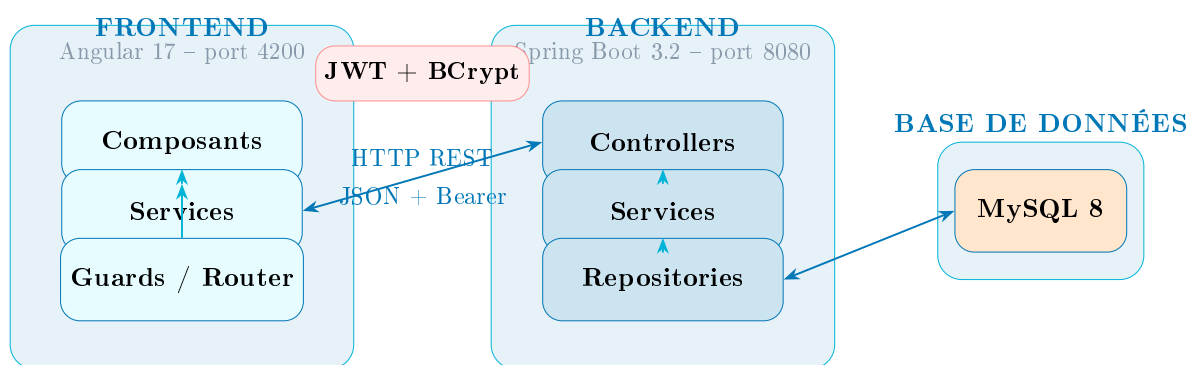


Figure 2.1: Architecture 3-tiers de l'application

2.2 Structure du Backend (Spring Boot)

Le backend est organisé en **4 couches** selon le patron d'architecture MVC :

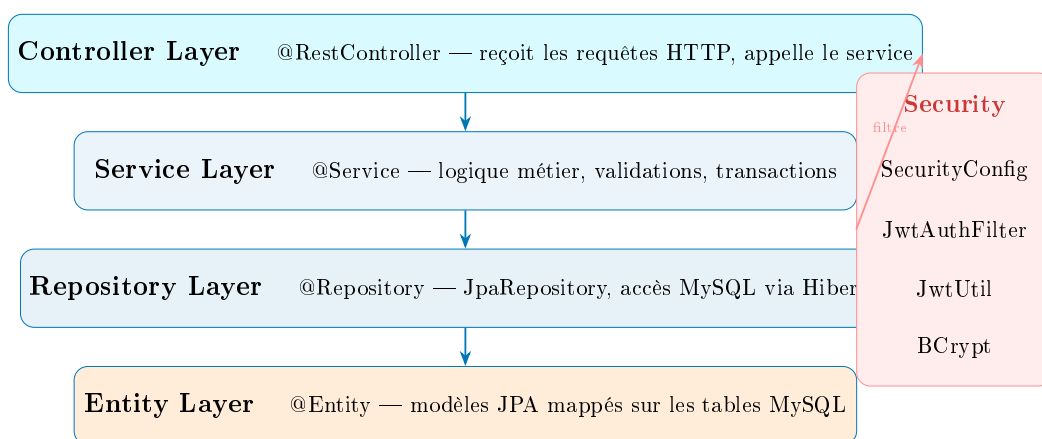


Figure 2.2: Architecture en couches du backend Spring Boot

2.3 Structure du Frontend (Angular)

Table 2.1: Organisation des modules Angular

darkblue	Contenu et rôle
core/services/	AuthService, FormationService, ThemeService — communication API
core/guards/	AuthGuard (connexion requise), RoleGuard (vérification du rôle)
core/interceptors/	JwtInterceptor — ajout automatique du Bearer token
features/auth/	Login, Register, VerifyEmail
features/landing/	Page d'accueil publique animée
features/dashboard/	Tableau de bord statistiques
features/formations/	CRUD formations + export Excel
features/participants/	CRUD + panel formations associées
shared/models/	Interfaces TypeScript (Formation, Participant...)

Chapter 3

Modèle de Données

3.1 Diagramme Entité-Association

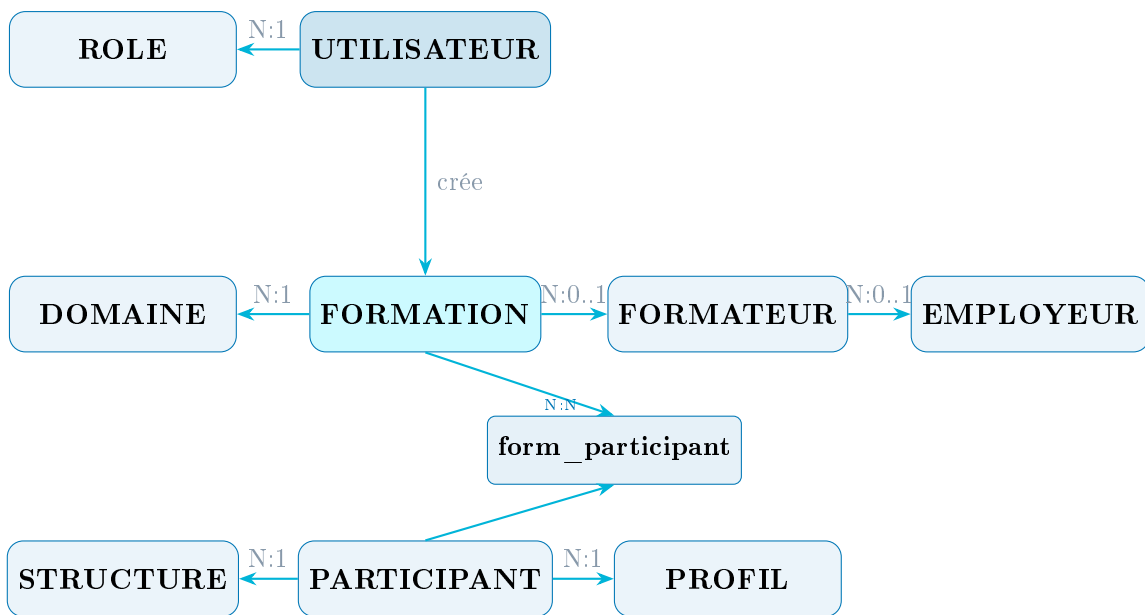


Figure 3.1: Diagramme Entité-Association simplifié

3.2 Description des Tables

Table 3.1: Tables principales de la base `gestion_formation`

darkblue	Attributs principaux	
role	id (PK), nom	3 valeurs : ADMIN, RESPONSABLE, UTILISATEUR
utilisateur	id, login (UK), password, email (UK), id_role (FK), email_verifie, actif, token_verification	Vérification email avant activation
formation	id, titre, annee, duree, budget, lieu, date_formation, id_domaine (FK), id_formateur (FK), created_by_id	Entité centrale du système
formation_participant	id_formation, id_participant	Table d'association N-N
participant	id, nom, prenom, email, tel, id_structure (FK), id_profil (FK)	Lié à structure et profil
formateur	id, nom, prenom, email, tel, type (INTERNE/EXTERNE), id_employeur (FK)	Distinction interne/externe
domaine	id, libelle (UK)	Ex: Informatique, Finance
structure	id, libelle (UK)	Direction centrale/régionale
profil	id, libelle (UK)	Ex: Informaticien Bac+5
employeur	id, nom_employeur (UK)	Pour formateurs externes

3.3 Script SQL de création

```

1 CREATE DATABASE IF NOT EXISTS gestion_formation
2   CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3 USE gestion_formation;
4
5 CREATE TABLE role (
6   id INT PRIMARY KEY AUTO_INCREMENT,
7   nom VARCHAR(50) NOT NULL UNIQUE
8 );
9 INSERT INTO role(nom) VALUES ('ADMIN'),('RESPONSABLE'),('
  UTILISATEUR');
10
11 CREATE TABLE utilisateur (
12   id INT PRIMARY KEY AUTO_INCREMENT,
13   login VARCHAR(100) NOT NULL UNIQUE,
14   password VARCHAR(255) NOT NULL,
15   email VARCHAR(150) UNIQUE,
16   id_role INT NOT NULL,
17   email_verifie BOOLEAN DEFAULT FALSE,
18   actif BOOLEAN DEFAULT FALSE,
19   token_verification VARCHAR(100),
20   FOREIGN KEY (id_role) REFERENCES role(id)

```

```
21 );
22
23 CREATE TABLE formation (
24     id                BIGINT PRIMARY KEY AUTO_INCREMENT,
25     titre              VARCHAR(200) NOT NULL,
26     annee              INT,  duree INT,  budget DOUBLE,
27     lieu               VARCHAR(200),
28     date_formation     DATE,
29     id_domaine         INT,
30     id_formateur       INT,
31     created_by_id      INT,
32     FOREIGN KEY (id_domaine) REFERENCES domaine(id),
33     FOREIGN KEY (id_formateur) REFERENCES formateur(id)
34 );
35
36 -- Relation N-N
37 CREATE TABLE formation_participant (
38     id_formation       BIGINT, id_participant INT,
39     PRIMARY KEY (id_formation, id_participant)
40 );
```

Listing 3.1: Extrait du script schema.sql

Chapter 4

Sécurité et Authentification

4.1 Flux d'Authentification JWT

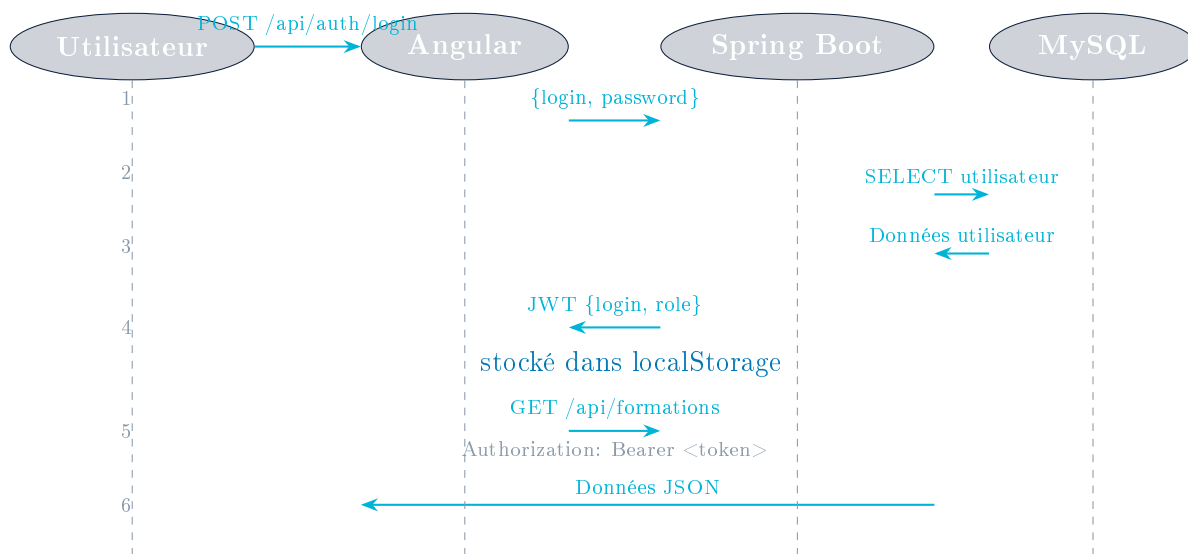


Figure 4.1: Diagramme de séquence — Authentification JWT

4.2 Implémentation JWT

```
1 @Component
2 public class JwtUtil {
3
4     @Value("${jwt.secret}")
5     private String secret;
6
7     // G\ '{e}n\ '{e}ration du token avec le login ET le r\ ^{o}le
8     public String generateToken(String username, String role) {
9         return Jwts.builder()
10             .setSubject(username)
11             .claim("role", role) // R\ ^{o}le inclus dans le
12                                 token
13             .setIssuedAt(new Date())
14             .setExpiration(new Date(System.currentTimeMillis() +
15                                     86400000L))
16             .signWith(getSigningKey(), SignatureAlgorithm.HS256)
17             .compact();
18     }
19 }
```

```

17
18 // Extraction du r\^ole depuis le token
19 public String extractRole(String token) {
20     return (String) getClaims(token).get("role");
21 }
22
23 public boolean isTokenValid(String token) {
24     try { getClaims(token); return true; }
25     catch (JwtException e) { return false; }
26 }
27 }

```

Listing 4.1: JwtUtil.java — Génération et validation du token

4.3 Configuration Spring Security

```

1 @EnableMethodSecurity(prePostEnabled = true)
2 public class SecurityConfig {
3
4     @Bean
5     public SecurityFilterChain filterChain(HttpSecurity http)
6         throws Exception {
7         http
8             .authorizeHttpRequests(auth -> auth
9                 // Public : login, register, v\^rification email
10                 .requestMatchers("/api/auth/**").permitAll()
11
12                 // Lecture pour tous les r\^oles (pour remplir les
13                 // formulaires)
14                 .requestMatchers(HttpMethod.GET, "/api/domaines/**").
15                     authenticated()
16                 .requestMatchers(HttpMethod.GET, "/api/structures/**").
17                     authenticated()
18
19                 // \^criture r\^serv\^e \^a ADMIN uniquement
20                 .requestMatchers("/api/utilisateurs/**").hasRole("ADMIN")
21                 .requestMatchers(HttpMethod.POST, "/api/domaines/**").
22                     hasRole("ADMIN")
23
24                 // RESPONSABLE + ADMIN peuvent g\^rer les formations
25                 .requestMatchers(HttpMethod.POST, "/api/formations/**")
26                     .hasAnyRole("ADMIN", "RESPONSABLE")
27
28                 // Tout le reste : \^tre connect\^ suffit
29                 .anyRequest().authenticated()
30             )
31     }
32     .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
33     .addFilterBefore(jwtAuthFilter,
34         UsernamePasswordAuthenticationFilter.class);
35     return http.build();
36 }

```

```
29 }  
30 }
```

Listing 4.2: SecurityConfig.java — Règles d'accès par endpoint

4.4 Inscription et Vérification Email

Le processus d'inscription en auto-service est protégé par plusieurs mécanismes :

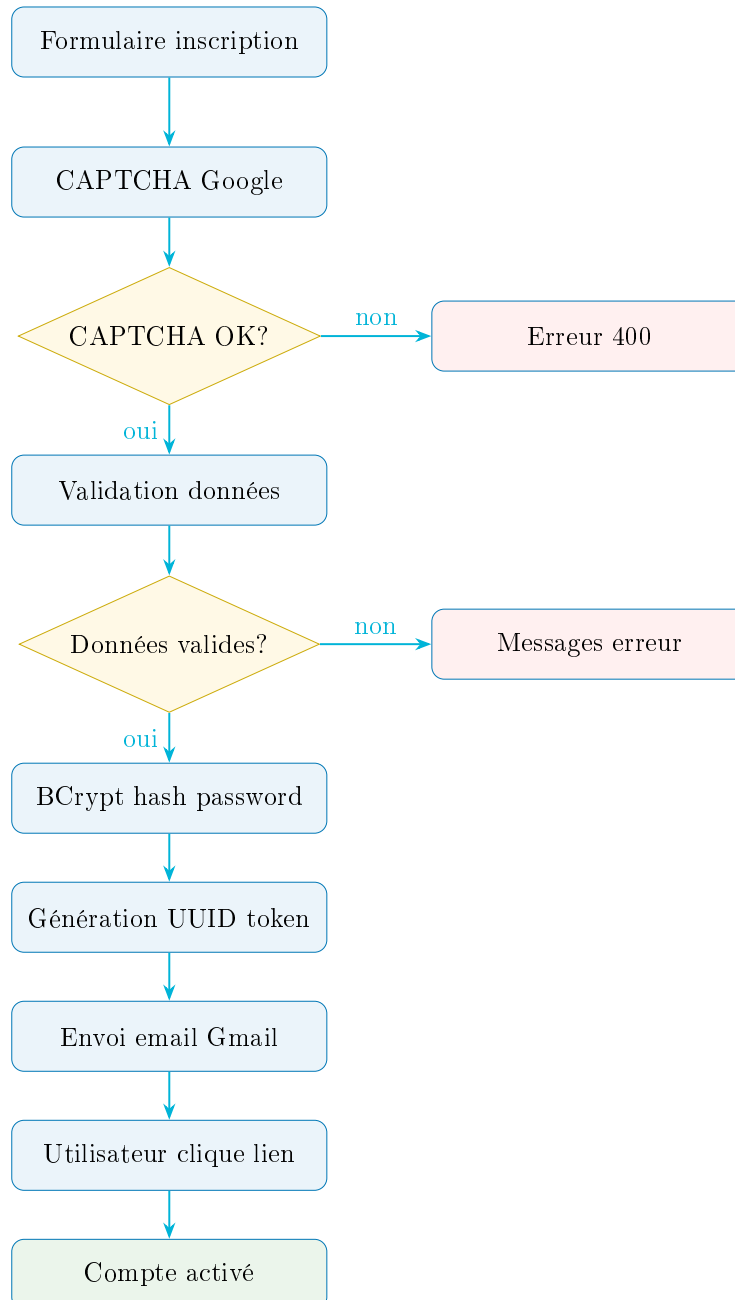


Figure 4.2: Flux d'inscription avec vérification email

Règles du mot de passe fort (inscription uniquement)

Minimum 8 caractères • Au moins 1 majuscule • Au moins 1 minuscule • Au moins 1 chiffre • Au moins 1 caractère spécial (!@#\$%^&*...)

Chapter 5

Gestion des Rôles (RBAC)

5.1 Les Trois Rôles

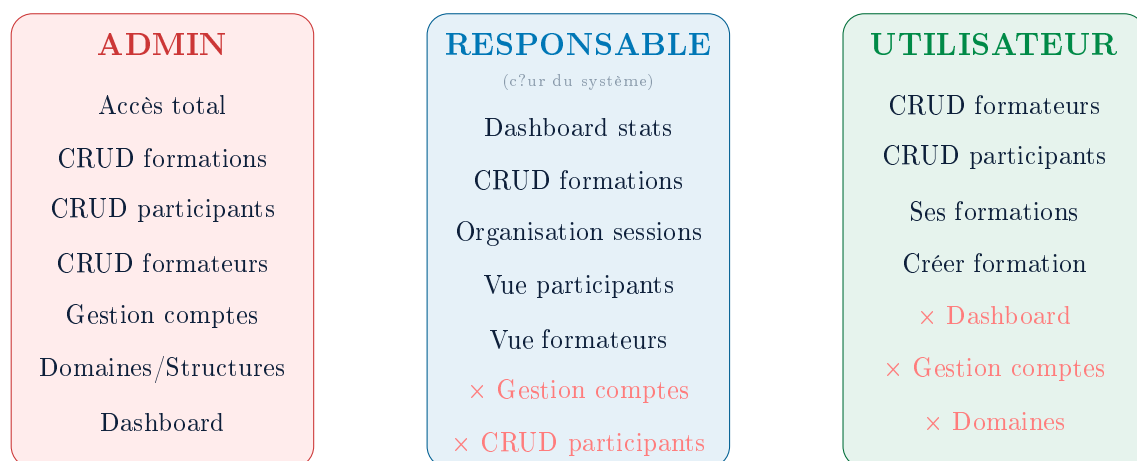


Figure 5.1: Permissions par rôle

5.2 Tableau des Permissions Détaillées

Table 5.1: Matrice des permissions RBAC

darkblue	ADMIN	RESPONSABLE	UTILISATEUR
Dashboard / statistiques			×
Voir formations			
CRUD formations			Ses propres
Organisation formation			×
CRUD participants		×	
CRUD formateurs		×	
Export Excel			×
Domaines (lecture)			
Domaines (écriture)		×	×
Gestion utilisateurs		×	×

5.3 Guards Angular

```
1 @Injectable({ providedIn: 'root' })
2 export class RoleGuard implements CanActivate {
```

```
3
4  canActivate(route: ActivatedRouteSnapshot): boolean {
5      // R\`{e}cup\`{e}re les r\`{o}les autoris\`{e}s depuis la
        configuration de la route
6      const allowedRoles: string[] = route.data['roles'] || [];
7      const userRole = this.auth.getRole(); // Depuis localStorage
8
9      if (allowedRoles.length === 0 || allowedRoles.includes(
        userRole!))
10         return true;
11
12         // Acc\`{e}s refus\`{e} ? page 403
13         this.router.navigate(['/access-denied']);
14         return false;
15     }
16 }
17
18 // Utilisation dans le routing :
19 { path: 'dashboard',
20   component: DashboardComponent,
21   canActivate: [RoleGuard],
22   data: { roles: ['ADMIN', 'RESPONSABLE'] } // UTILISATEUR exclu
23 }
```

Listing 5.1: RoleGuard — Protection des routes par rôle

Chapter 6

Implémentation Technique

6.1 Entités JPA

```
1 @Entity
2 @Table(name = "formation")
3 @Data // Lombok : g\ '{e}n\ '{e}re getters/
   setters
4 @NoArgsConstructor // Lombok : constructeur vide requis par
   JPA
5 public class Formation {
6
7     @Id
8     @GeneratedValue(strategy = GenerationType.IDENTITY)
9     private Long id;
10
11     @NotBlank(message = "Le titre est obligatoire")
12     @Size(min = 3, max = 200, message = "3 \ '{a} 200 caract\ '{e}
        res")
13     @Column(nullable = false, length = 200)
14     private String titre;
15
16     @NotNull(message = "L'ann\ '{e}e est obligatoire")
17     @Min(value = 2000)
18     @Max(value = 2100)
19     private Integer annee;
20
21     @NotNull(message = "La dur\ '{e}e est obligatoire")
22     @Min(value = 1, message = "Minimum 1 jour")
23     @Max(value = 365)
24     private Integer duree;
25
26     @DecimalMin(value = "0.0", message = "Budget non n\ '{e}gatif"
        )
27     private Double budget;
28
29     // Relations JPA (cl\ '{e}s \ '{e}trang\ '{e}res)
30     @NotNull(message = "Le domaine est obligatoire")
31     @ManyToOne(fetch = FetchType.EAGER)
32     @JoinColumn(name = "id_domaine")
33     private Domaine domaine;
34
35     @ManyToMany(fetch = FetchType.EAGER)
36     @JoinTable(name = "formation_participant",
```

```

37     joinColumns = @JoinColumn(name = "id_formation"),
38     inverseJoinColumns = @JoinColumn(name = "id_participant")
39     )
40     private Set<Participant> participants = new HashSet<>();
41 }

```

Listing 6.1: Formation.java — Entité principale avec validation

6.2 Export Excel avec Apache POI

```

1  @GetMapping("/export/excel")
2  public ResponseEntity<byte[]> exportExcel() throws Exception {
3      List<Formation> formations = service.findAll();
4
5      try (XSSFWorkbook wb = new XSSFWorkbook();
6           ByteArrayOutputStream out = new ByteArrayOutputStream())
7      {
8          XSSFSheet sheet = wb.createSheet("Formations");
9
10         // En-tête stylisé : fond bleu indigo, texte blanc, bold
11         XSSFCellStyle headerStyle = wb.createCellStyle();
12         headerStyle.setFillForegroundColor(
13             new XSSFCColor(new byte[]{63,81,(byte)181}, null));
14         headerStyle.setFillPattern(FillPatternType.SOLID_FOREGROUND);
15         XSSFFont headerFont = wb.createFont();
16         headerFont.setColor(new XSSFCColor(
17             new byte[]{(byte)255,(byte)255,(byte)255}, null));
18         headerFont.setBold(true);
19         headerStyle.setFont(headerFont);
20
21         // Colonnes : ID, Titre, Ann\{e}, Dur\{e}, Budget,
22         //             Formateur, Lieu, Date, Nb Participants,
23         //             Participants
24         String[] headers = {"ID","Titre","Ann\{e}","Dur\{e}",
25                             "Budget",
26                             "Domaine","Formateur","Lieu","Date","
27                             Participants"};
28
29         // ... création des cellules ...
30
31         wb.write(out);
32         return ResponseEntity.ok()
33             .header(HttpHeaders.CONTENT_DISPOSITION,
34                     "attachment; filename=formations.xlsx")
35             .contentType(MediaType.parseMediaType(
36                 "application/vnd.openxmlformats-officedocument.
37                 spreadsheetml.sheet"))

```

```
33         .body(out.toByteArray());
34     }
35 }
```

Listing 6.2: FormationController.java — Endpoint d'export Excel

6.3 Service Angular avec RxJS

```
1  @Injectable({ providedIn: 'root' })
2  export class FormationService {
3      private api = 'http://localhost:8080/api/formations';
4
5      constructor(private http: HttpClient) {}
6
7      // Récupère toutes les formations (Observable =
8      // promesse améliorée)
9      getAll(): Observable<Formation[]> {
10         return this.http.get<Formation[]>(this.api);
11     }
12
13     // Exporte en Excel (réponse Blob = fichier binaire)
14     exportExcel(): Observable<Blob> {
15         return this.http.get(`${this.api}/export/excel`, {
16             responseType: 'blob' });
17     }
18
19     // Statistiques pour les graphiques
20     getStatsByDomaine(): Observable<{[k:string]:number}> {
21         return this.http.get<any>(`${this.api}/stats/domaine`);
22     }
23 }
24
25 // Utilisation dans le composant :
26 this.service.getAll().subscribe((data: Formation[]) => {
27     this.items = data;
28     this.applyFilters(); // Filtrage local
29 });
```

Listing 6.3: formation.service.ts — Communication avec l'API

Chapter 7

Contrôles de Saisie et Validation

7.1 Validation Backend (Jakarta Bean Validation)

Table 7.1: Règles de validation par champ

darkblue	Annotation	
nom / prénom	@NotBlank @Size @Pattern	Obligatoire, 2-100 car., lettres uniquement
email	@Email	Format valide avec @ et .
téléphone	@Pattern	8 à 15 chiffres numériques
titre formation	@NotBlank @Size	Obligatoire, 3-200 caractères
année	@Min(2000) @Max(2100)	Entre 2000 et 2100
durée	@Min(1) @Max(365)	1 à 365 jours
budget	@DecimalMin("0.0")	Valeur non négative
login	@NotBlank @Size	3-50 car., alphanumérique
mot de passe inscription	Validation métier	8+ car., majuscule, chiffre, spécial

7.2 Validation Frontend (Angular)

La validation côté Angular fonctionne en temps réel sur l'événement blur (perte de focus) :

```
1 validateField(field: string): string {
2   const v = (this.current as any)[field];
3   switch(field) {
4     case 'email':
5       if (v && !/^[\s@]+@[\s@]+\.[\s@]+$/\.test(v))
6         return 'Format invalide (doit contenir @ et .)';
7       return ''; // Pas d'erreur
8
9     case 'tel':
10      if (v && !/^[\+]?[0-9]{8,15}$/\.test(v.replace(/\s/g, '')))
11        return 'Num\''{e}ro invalide (8-15 chiffres)';
12      return '';
13   }
14 }
```

Listing 7.1: Exemple de validation Angular dans participants.component.ts

Double validation : Frontend ET Backend

Les contrôles sont appliqués **deux fois** : d'abord côté Angular (expérience utilisateur, réponse immédiate), puis côté Spring Boot (sécurité garantie même si le frontend est contourné). Cette approche respecte le principe de défense en profondeur.

Chapter 8

Interface Utilisateur

8.1 Design System

L'interface utilise un design system cohérent basé sur Angular Material avec un thème personnalisé :

Table 8.1: Palette de couleurs du design system

darkblue	Hex	
[HTML]050D1F	#050d1f	Fond principal (dark mode)
[HTML]0077B6	#0077b6	Titres, accents
[HTML]00B4D8	#00b4d8	Boutons, liens actifs
[HTML]00E5FFCyan	#00e5ff	Highlights, badges
[HTML]FFD60AOr	#ffd60a	Rôle Responsable
[HTML]00E676Vert	#00e676	Rôle Utilisateur, succès
[HTML]FF4444	#ff4444	Erreurs, rôle Admin

8.2 Pages et Navigation

Table 8.2: Pages de l'application et leur accès

darkblue	URL	
Accueil	/	Public — Landing animée avec dark/light mode
Connexion	/login	Public — Formulaire JWT
Inscription	/register	Public — CAPTCHA + validation mot de passe fort
Vérification email	/verify-email	Public — Activation compte
Dashboard	/app/dashboard	ADMIN + RESPONSABLE
Formations	/app/formations	Tous — CRUD selon rôle
Participants	/app/participants	ADMIN + UTILISATEUR
Formateurs	/app/formateurs	ADMIN + UTILISATEUR
Domaines	/app/domaines	ADMIN — CRUD
Utilisateurs	/app/utilisateurs	ADMIN uniquement
Accès refusé	/access-denied	Authentifié — Page 403

8.3 Fonctionnalités Spéciales

8.3.1 Mode Sombre / Mode Clair

Le `ThemeService` gère la bascule dark/light mode persistée en `localStorage`. Les variables CSS sont redéfinies pour chaque thème, permettant une transition fluide sans rechargement de page.

8.3.2 Tableau de Bord Statistique

Le dashboard (ADMIN + RESPONSABLE) affiche trois graphiques via **ng2-charts** (wrapper Angular de Chart.js) :

- **Graphique en barres** : Nombre de formations par domaine
- **Graphique en ligne** : Évolution du nombre de formations par année
- **Graphique doughnut** : Budget total par domaine (en DT)

8.3.3 Panel Formations par Participant

Dans la liste des participants, un **bouton expandable** affiche toutes les formations associées à chaque participant, avec titre, domaine, lieu, date et formateur.

Chapter 9

Tests et Validation Fonctionnelle

9.1 Scénarios de Test

Table 9.1: Tests fonctionnels réalisés

darkblue	Résultat	
Connexion ADMIN	OK	Token JWT généré, redirigé dashboard
Connexion UTILISATEUR	OK	Redirigé /app/formations (pas dashboard)
UTILISATEUR accède dashboard	OK	RoleGuard bloque, redirigé /access-denied
Formulaire email invalide	OK	Message d'erreur instantané sous le champ
Téléphone non numérique	OK	Erreur : "8-15 chiffres numériques"
Mot de passe faible	OK	Barre rouge + indicateurs non cochés
Export Excel	OK	Fichier .xlsx téléchargé avec mise en forme
RESPONSABLE modifie formation	OK	Autorisé (RESPONSABLE + ADMIN)
UTILISATEUR modifie formation	OK	Ses propres formations uniquement

9.2 Endpoints API testés

Table 9.2: Endpoints principaux et leur comportement

darkblue Méthode		Status	
POST	/api/auth/login	200 / 401	Token JWT ou erreur
POST	/api/auth/register	200 / 400	Succès ou erreur validation
GET	/api/auth/verify?token=...	200 / 400	Activation compte
GET	/api/formations	200	Liste JSON des formations
POST	/api/formations	200 / 400	Formation créée ou erreur validation
GET	/api/formations/export/excel	200	Fichier .xlsx
GET	/api/formations/stats/domaine	200	Map<String, Long>

Chapter 10

Guide de Déploiement

10.1 Prérequis

- Java JDK 17 (Eclipse Temurin recommandé)
- Node.js v20+ et npm
- MySQL Server 8.x
- IntelliJ IDEA (backend) et VS Code (frontend)
- Angular CLI : `npm install -g @angular/cli`

10.2 Configuration

```
1 # Base de donn\'}{e}s
2 spring.datasource.url=jdbc:mysql://localhost:3306/
   gestion_formation
3 spring.datasource.username=root
4 spring.datasource.password=votre_mot_de_passe
5
6 # JPA/Hibernate
7 spring.jpa.hibernate.ddl-auto=update
8 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.
   MySQLDialect
9
10 # JWT
11 jwt.secret=votreCleSecrete512BitsMinimum
12 jwt.expiration=86400000
13
14 # Email Gmail SMTP
15 spring.mail.host=smtp.gmail.com
16 spring.mail.port=587
17 spring.mail.username=votre.email@gmail.com
18 spring.mail.password=votre_app_password_gmail
19
20 # reCAPTCHA Google v2
21 recaptcha.secret=votre_cle_secrete_recaptcha
22 app.url=http://localhost:4200
```

Listing 10.1: application.properties — Configuration principale

10.3 Démarrage

```
1  -- 1. Cr\'}er la base de donn\'}es
2  mysql -u root -p < database/schema.sql
3
4  -- 2. D\'}marrer le backend
5  cd backend
6  mvn spring-boot:run
7  # API disponible sur http://localhost:8080
8
9  -- 3. D\'}marrer le frontend
10 cd frontend
11 npm install
12 ng serve
13 # Application disponible sur http://localhost:4200
```

Listing 10.2: Commandes de démarrage

Comptes de test

admin / admin123 (ADMIN) • **responsable** / admin123 (RESPONSABLE)
• **utilisateur** / admin123 (UTILISATEUR)

Chapter 11

Captures d'Écran de l'Application

11.1 Page d'Accueil (Landing Page)

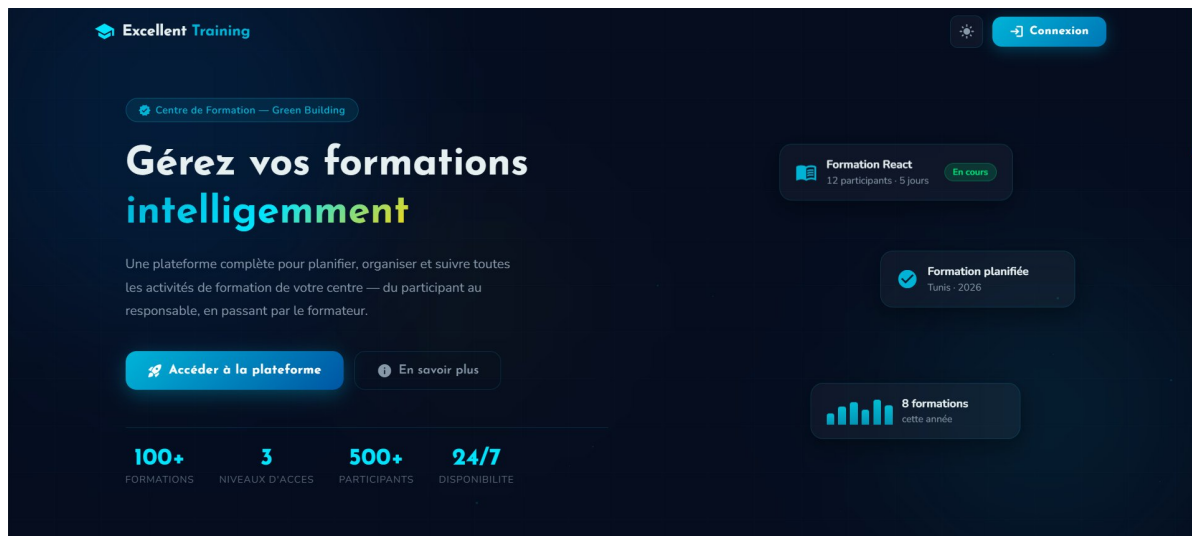


Figure 11.1: Page d'accueil publique — Dark mode avec cartes animées et bouton Connexion

Description de la page d'accueil

La landing page présente le centre Excellent Training avec un design dark moderne. Elle comprend un hero animé avec des statistiques (100+ formations, 3 niveaux d'accès, 500+ participants, 24/7), des cartes flottantes simulant l'activité, et un bouton Connexion. Un toggle dark/light mode est disponible en haut à droite.

11.2 Pages d'Authentification

11.2.1 Page de Connexion

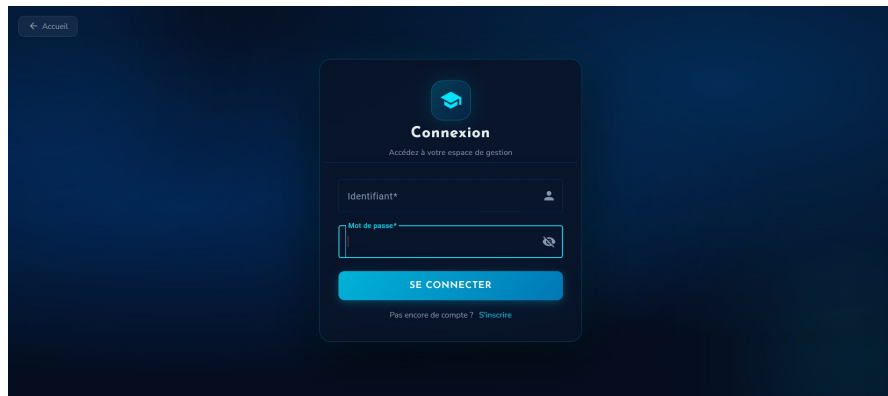
The image shows a login page with a dark blue background. At the top left, there is a link '← Accueil'. In the center, there is a white box containing a graduation cap icon, the title 'Connexion', and the subtitle 'Accédez à votre espace de gestion'. Below this, there are two input fields: 'Identifiant*' and 'Mot de passe*'. The password field has a toggle icon for visibility. A blue button labeled 'SE CONNECTER' is positioned below the fields. At the bottom of the white box, there is a link 'Pas encore de compte ? S'inscrire'.

Figure 11.2: Page de connexion — Formulaire sécurisé avec JWT

11.2.2 Page d'Inscription avec CAPTCHA

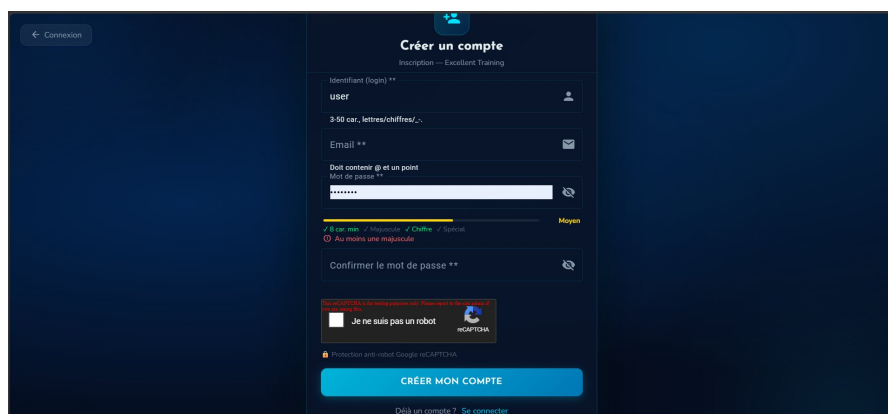
The image shows a registration page with a dark blue background. At the top left, there is a link '← Connexion'. In the center, there is a white box containing a graduation cap icon, the title 'Créer un compte', and the subtitle 'Inscription — Excellent Training'. Below this, there are four input fields: 'Identifiant (login) **', 'Email **', 'Mot de passe **', and 'Confirmer le mot de passe **'. The password field has a strength indicator bar showing 'Moyen'. Below the password fields, there is a Google reCAPTCHA v2 widget with the text 'Je ne suis pas un robot'. A blue button labeled 'CRÉER MON COMPTE' is positioned below the fields. At the bottom of the white box, there is a link 'Déjà un compte ? Se connecter'.

Figure 11.3: Page d'inscription — Validation en temps réel, barre de force, Google reCAPTCHA v2

Fonctionnalités de la page d'inscription

- Validation en temps réel sur chaque champ (blur)
- Barre de force du mot de passe : Faible ? Moyen ? Fort ? Très fort
- Indicateurs visuels : 8 car. min ?, Majuscule ?, Chiffre ?, Spécial ?
- Google reCAPTCHA v2 intégré et vérifié côté backend
- Lien vers la page de connexion pour les utilisateurs existants

11.2.3 Email de Vérification

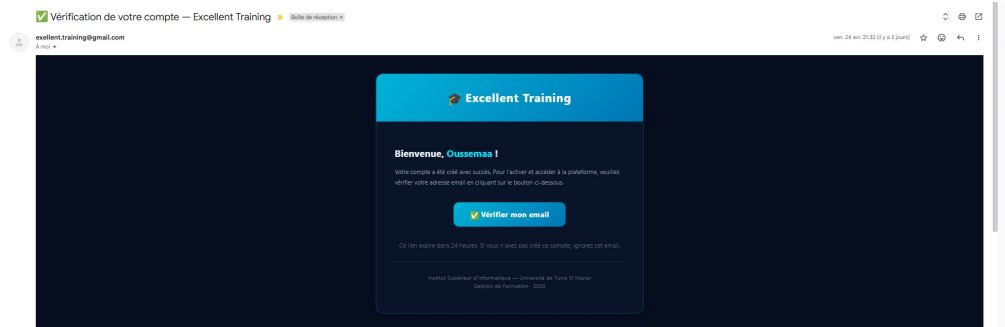


Figure 11.4: Email HTML de vérification reçu dans la boîte Gmail

L'email est envoyé via **Gmail SMTP** (JavaMail + Spring Boot Starter Mail) avec un design HTML cohérent au thème de l'application. Il contient un lien de vérification unique (UUID) expirant dans 24 heures.

11.3 Tableau de Bord (Admin)

11.3.1 Vue d'ensemble et Statistiques

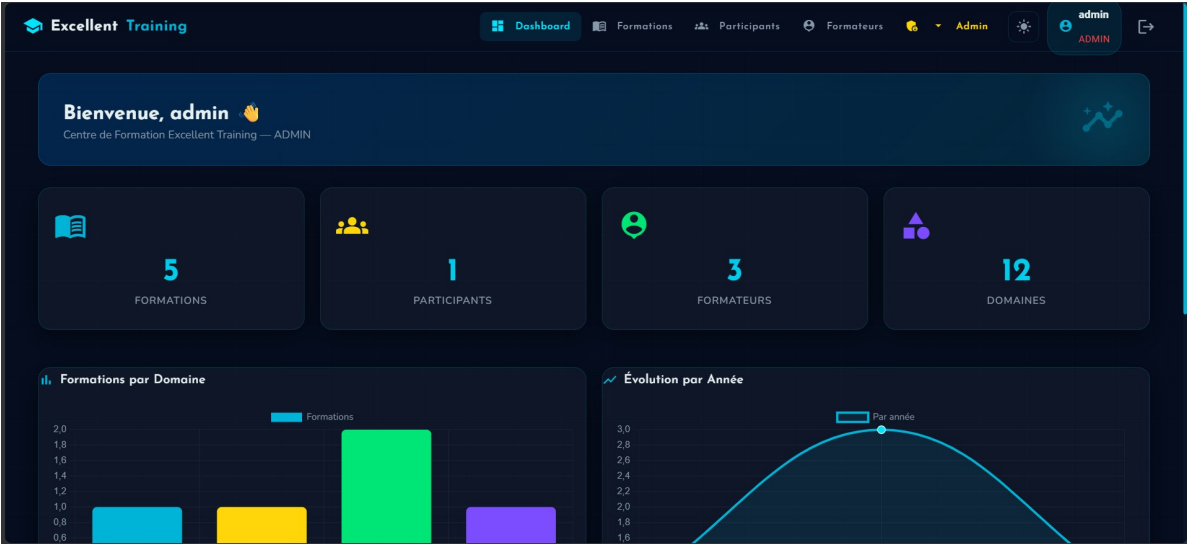


Figure 11.5: Dashboard Admin — Bannière de bienvenue et 4 cartes de statistiques

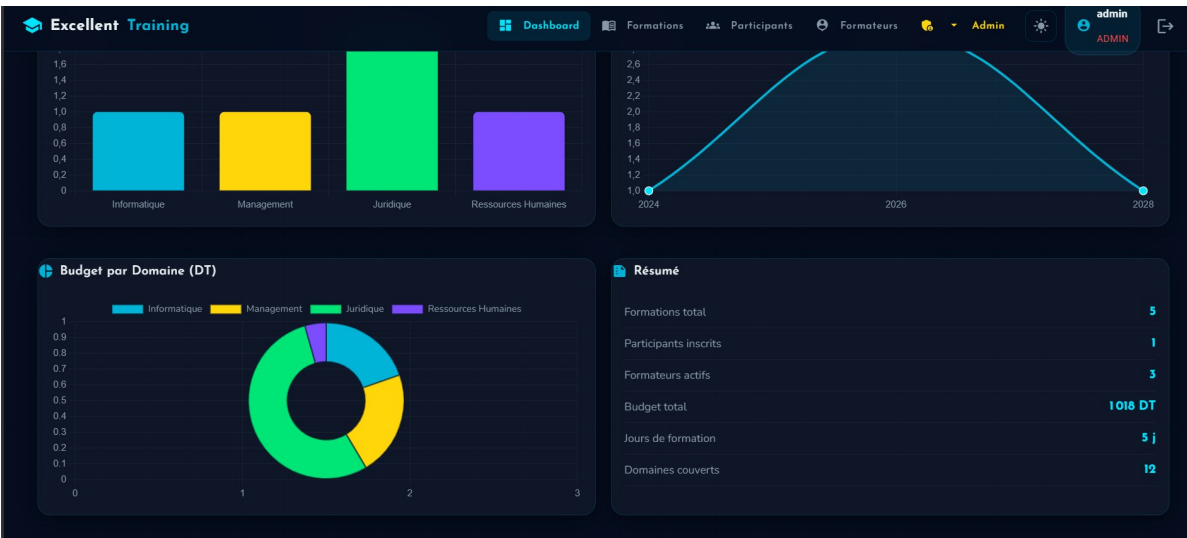


Figure 11.6: Dashboard Admin — Graphiques : barres (domaines), ligne (années), doughnut (budget) et résumé

Graphiques du tableau de bord

- **Formations par Domaine** : Graphique en barres colorées (Informatique, Management, Juridique, RH...)
- **Évolution par Année** : Graphique en ligne avec remplissage (ng2-charts / Chart.js)
- **Budget par Domaine** : Graphique doughnut avec légende
- **Résumé** : Tableau récapitulatif (formations, participants, formateurs, budget total, jours, domaines)

11.3.2 Barre de Navigation Admin

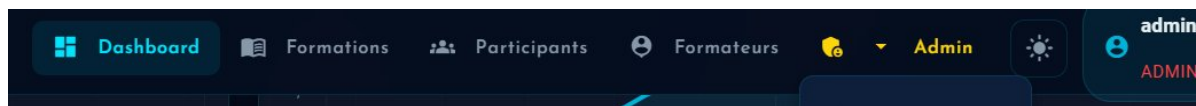


Figure 11.7: Navbar Admin — Dashboard, Formations, Participants, Formateurs, menu Admin (en or)

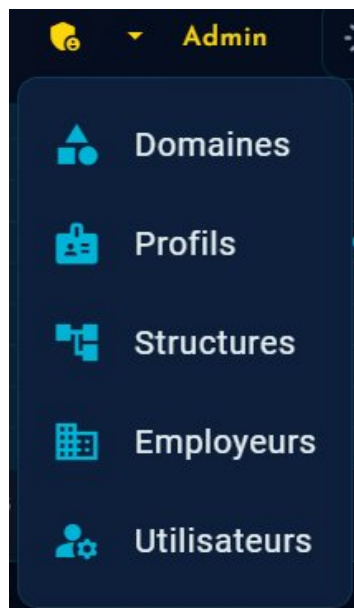


Figure 11.8: Menu déroulant Admin — Domaines, Profils, Structures, Employeurs, Utilisateurs

11.4 Dashboard Responsable

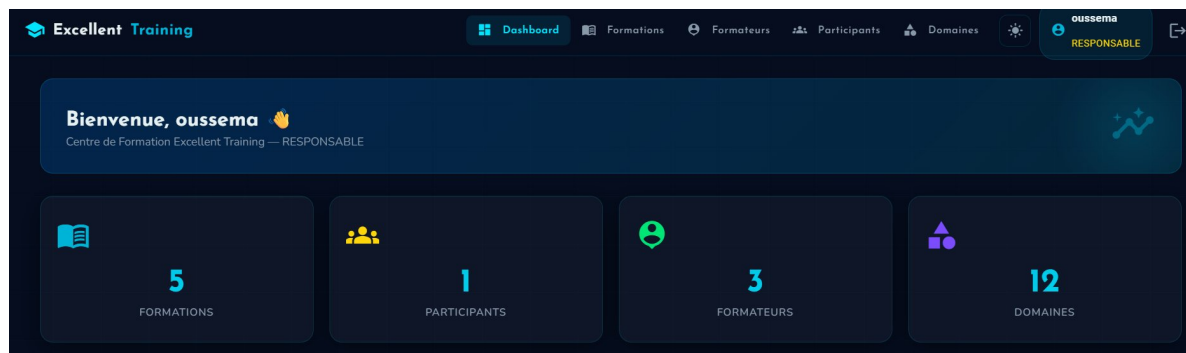


Figure 11.9: Dashboard Responsable — Mêmes statistiques que l'Admin, sans menu de gestion des comptes

Différence Admin vs Responsable

Le Responsable voit le même dashboard que l'Admin (statistiques, graphiques) mais sans le menu **Admin** en or dans la navbar. Il n'a pas accès à la gestion des utilisateurs, domaines, structures, profils et employeurs.

11.5 Interface Utilisateur Simple

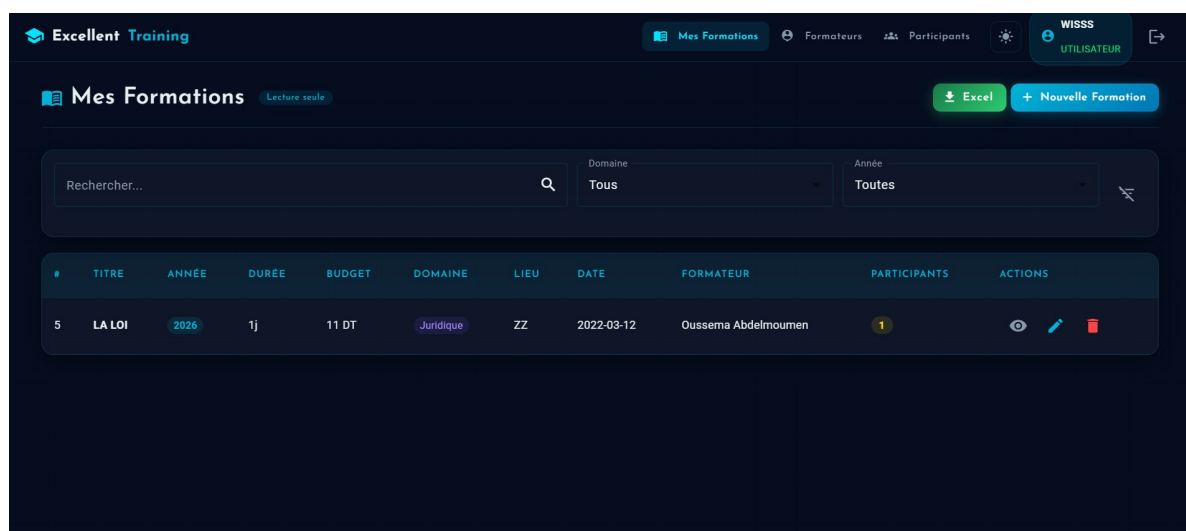


Figure 11.10: Interface Utilisateur — "Mes Formations" (lecture seule), Formateurs, Participants. Pas de Dashboard.

Contrôle d'accès côté interface

- **Navbar simplifiée** : Seulement Mes Formations, Formateurs, Participants
- **Badge "Lecture seule"** : Affiché sur la page formations pour l'UTILISATEUR
- **Boutons contextuels** : "Nouvelle Formation" visible (ses propres formations) + Export Excel
- **Pas de Dashboard** : RoleGuard bloque l'accès et redirige vers /access-denied
- **Filtrage des données** : L'UTILISATEUR ne voit que ses formations (created_by_id)

Chapter 12

Conclusion

12.1 Bilan du Projet

Ce projet a permis de réaliser une application web complète répondant à toutes les exigences du cahier des charges [1]. Les principaux acquis sont :

- Maîtrise de l'architecture **Spring Boot** : entités JPA, repositories, services, controllers REST
- Implémentation d'une **sécurité JWT** robuste avec gestion des rôles (RBAC)
- Développement d'une **SPA Angular** avec navigation protégée par guards
- Mise en place de **contrôles de saisie** à deux niveaux (frontend + backend)
- Intégration de fonctionnalités avancées : **export Excel**, **vérification email**, **reCAPTCHA**
- Design moderne avec **dark/light mode** et animations CSS

12.2 Perspectives d'Amélioration

- Génération de rapports PDF (JasperReports ou iText)
- Notifications en temps réel (WebSocket)
- Application mobile complémentaire (React Native ou Ionic)
- Déploiement sur cloud (Docker + AWS/Azure)
- Tests unitaires et d'intégration (JUnit + Karma/Jasmine)

Dépôt GitHub du projet

<https://github.com/oussema-abdelmoumen/Gestion-Formation-Excellent-Training>

Stack : Spring Boot 3.2 • Angular 17 • MySQL 8 • JWT • BCrypt • Apache POI • reCAPTCHA v2

Bibliography

- [1] Institut Supérieur d'Informatique. *Cahier des charges — Mini-Projet 2026 : Conception et développement d'une application de gestion de formation*. ISI Tunis El Manar, 2025.
- [2] Spring Framework Documentation. *Spring Boot Reference Documentation 3.2.x*. Pivotal Software, 2024. <https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/>
- [3] Google. *Angular Documentation v17*. Google, 2024. <https://angular.io/docs>
- [4] Auth0. *JSON Web Tokens — Introduction*. Auth0, 2024. <https://jwt.io/introduction>
- [5] Spring Security Documentation. *Spring Security Reference 6.2*. Pivotal, 2024. <https://docs.spring.io/spring-security/reference/>
- [6] Apache Software Foundation. *Apache POI — the Java API for Microsoft Documents*. Apache, 2024. <https://poi.apache.org/>
- [7] Provos N., Mazières D. *A Future-Adaptable Password Scheme*. USENIX Annual Technical Conference, 1999.
- [8] Google. *reCAPTCHA v2 Developer Guide*. Google Cloud, 2024. <https://developers.google.com/recaptcha/docs/display>
- [9] Red Hat. *Hibernate ORM 6.3 User Guide*. Red Hat, 2024. <https://docs.jboss.org/hibernate/orm/6.3/>